Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Billing, metering and customer service for electricity, water and gas

Mid-Semester Project Report

Group number	Group 3
Course and section	Software Engineering & Project Management (SEPM), CSEF
Date	October 2026

#	Name	Registration number
1	Sumit P Surendran	20243590
2	Sreethan Reddy	20243550
3	Sushrut Lad	20243551
4	Viswajith	20243559

Contents

1. Problem statement and motivation · 2. Objectives · 3. Process model and feasibility study · 4. Scope · 5. Stakeholders · 6. Requirements · 7. Use cases and user stories · 8. System design · 9. Technology stack · 10. Implementation progress · 11. Testing · 12. Project management · 13. Individual contribution · 14. Challenges · Appendix

The SRS, use cases, full design document, UML diagrams, UI screens, test cases and project-management records are also in folders 02–06 of the ZIP as separate PDFs.

1. Problem Statement and Motivation

Many small utility offices and housing societies in India still handle electricity, water and gas billing on paper. A meter reader writes readings in a register, someone in the office types them into a spreadsheet, and the bill is worked out by hand from a tariff table. Going through sample bills and the way these offices usually work, the same problems kept coming up: slab rates get applied wrongly, a typo in a reading gives a huge (or negative) bill, payments can only be made at a counter, complaints get lost, and the manager has no quick way to see how much money is still due.

Problem statement: build a web application that keeps customers, connections, meters, readings, bills, payments, complaints and outages for all three utilities in one database, with separate views for the admin, employees and customers.

This topic was given to us for our DBMS project. Since it is a proper multi-user system with real business rules, we decided to build the same system for SEPM too. The DBMS side looks at the schema, triggers and views; this report is about how we engineered it, phase by phase.

2. Project Objectives

1. Let employees enter meter readings in a form that shows the last reading and refuses impossible values.
2. Generate the bill automatically from each reading using slab tariffs, a fixed charge and tax.
3. Give customers a portal to see their bills and usage, pay online and raise complaints.
4. Track complaints and outages, and tell affected customers automatically.
5. Give the admin a dashboard with revenue, dues, complaints and consumption by area.
6. Keep the data correct with database constraints and triggers, and keep each user to their own pages.
7. Follow the Waterfall model properly, with a document at the end of each phase.

3. Process Model and Feasibility Study

3.1 Why Waterfall

We went with the Waterfall model. Our requirements were clear from the start (metering and slab billing follow fixed rules), the DBMS course needed the full database design before any coding, and the semester has fixed checkpoints that line up well with finishing phases. Most of us are also new to web development, so a clear order of steps was easier to follow than sprints. To reduce the usual Waterfall risk of finding mistakes late, we reviewed each phase's output as a team before moving on, and wrote tests while coding instead of leaving them all for the end.

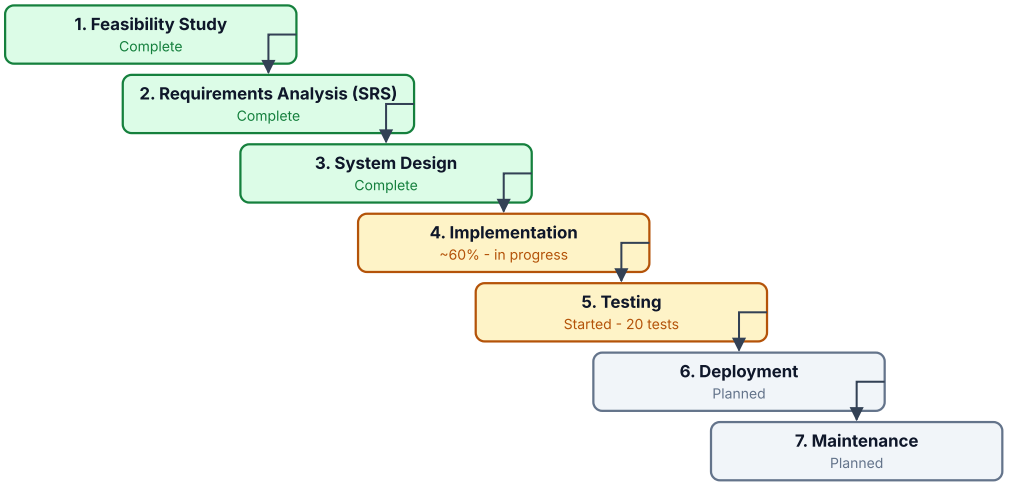


Figure 1: Waterfall phases and where we are at mid-semester

3.2 Feasibility study

Type	What we checked	Result
Technical	Needs only Python, Flask, SQLite and a browser. All free, run on our laptops, no database server. SQLite supports keys, CHECK constraints, triggers, views and indexes. We already know Python and SQL; Flask took a few days to pick up.	Feasible
Economic	Software and hardware cost nothing (open source, own laptops). Payment gateway is simulated. Effort is about 4 people × 6 hours/week × 14 weeks ≈ 336 hours. For a real office the savings come from fewer billing errors and less re-typing.	Feasible
Operational	Staff and customers already use phones and UPI apps. One task per page, familiar layout, error messages in plain words. Little training needed.	Feasible
Schedule	About 4 weeks for feasibility, requirements and design, 6 for coding, 4 for testing and wrap-up. The core was done by mid-sem.	Feasible
Legal	Only made-up sample data. Passwords are hashed and customers only see their own data. A real version would need to follow the DPDP Act 2023 and RBI rules for payments.	Feasible for a prototype

Table 1: Feasibility summary

4. Scope of the Project

4.1 Included

- Login with three roles: admin, employee, customer
- Customer registration, search and profile
- Connections and meters for electricity, water and gas
- Meter reading entry with checks
- Automatic slab billing and printable bills
- Online (simulated), counter and part payments
- Complaints, outages and in-app notifications
- Tariff editing, employees, dashboards and reports

4.2 Excluded

- Real payment gateway (payments are simulated)
- Smart meters / automatic readings
- SMS and e-mail alerts
- Mobile app (the site works on phones instead)
- Late fees, GST invoices, accounting export

4.3 Assumptions and constraints

- One connection per utility per customer, one active meter per connection
- Meters are cumulative, so readings only go up
- Tariff depends on utility and category (residential or commercial)
- Bills are due 15 days after the bill date
- Free software only, must run on a student laptop, one semester

5. Stakeholder Identification

Stakeholder	What they need	Where they use the system
Admin (utility manager)	Control over tariffs and staff, a view of revenue and dues	Dashboard, tariffs, employees, reports
Meter reader	Quick reading entry without mistakes	Add Meter Reading
Billing officer	Correct bills, record cash payments, add customers	Customers, connections, billing, payments
Technician	See and close complaints and outages	Complaints, outages
Customer	See and pay bills, track usage, report problems	Customer portal
Instructor / evaluator	Proof of SEPM and DBMS practice, a working demo	Report, demo, code
Regulator, bank (external)	Tariffs applied correctly, safe payments	Indirect

6. Requirements Specification

We collected requirements from the project brief, sample electricity and water bills, published slab tariffs, a few existing self-service portals, and by working through what each type of user needs. The full SRS is in `02_Requirements/SRS.pdf`.

6.1 Functional requirements

ID	Area	Requirement	Priority	Status
FR-01	Login	Users log in with username/email and password; passwords are stored hashed.	Must	Done
FR-02	Access	Each page only opens for its roles; others get "403", and logged-out users go to the login page.	Must	Done
FR-03	Customers	Staff add a customer (name, unique email, 10-digit phone, address, area, category, password).	Must	Done
FR-04	Customers	Staff search customers and open a profile with their connections and bills.	Should	Done
FR-05	Connections	Staff add a connection; the tariff is picked from the category and a meter is created. One per utility per customer.	Must	Done
FR-06	Readings	Picking a meter shows the customer, utility and last reading; consumption is shown while typing.	Must	Done
FR-07	Readings	Readings lower than the last one, or not after its date, are refused.	Must	Done
FR-08	Billing	Saving a reading creates the bill: slab charges + fixed charge + tax, itemised, due in 15 days.	Must	Done
FR-09	Billing	Staff list and filter bills; anyone allowed can open a bill and print it.	Must	Done
FR-10	Payments	Customers pay online by card, UPI, net banking or wallet (simulated), fully or in part, and get a receipt number.	Must	Done
FR-11	Payments	Paying more than is due is refused; the bill becomes partial or paid automatically.	Must	Done
FR-12	Payments	Staff record cash payments and see all payments with totals by method.	Should	Done
FR-13	Complaints	Customers raise complaints; staff assign, prioritise, update and resolve them.	Must	Done
FR-14	Outages	Staff report outages by utility and area (scheduled or active) and mark them resolved.	Should	Done
FR-15	Notifications	Customers get in-app notices for new bills, payments, outages in their area and complaint updates.	Should	Done
FR-16	Tariffs	Admin edits slab rates, fixed charge and tax; only new bills use the new values.	Must	Done
FR-17	Employees	Admin sees employees and their workload and can add new ones.	Should	Done
FR-18	Dashboards	Staff and customer dashboards with key numbers, charts and recent items.	Should	Done
FR-19	Reports	Usage per utility, by area and by category, collection rate and overdue bills.	Could	Partial
FR-20	Portal	Customers see connections, usage history and local outages, and edit phone, address and password.	Should	Done
FR-21	Billing	Late-payment charge and overdue reminders.	Could	Planned
FR-22	Billing	Server-generated PDF invoice and e-mail.	Could	Planned
FR-23	Meters	Replace a faulty meter and start the new one from its own reading.	Should	Planned
FR-24	Reports	Export reports to CSV/Excel.	Could	Planned
FR-25	Payments	Real payment gateway.	Won't	Planned

6.2 Non-functional requirements

ID	Type	Requirement	How we meet it
NFR-01	Security	Passwords are never stored as plain text	Salted hashes from Werkzeug
NFR-02	Privacy	Customers only see their own records	Queries filtered by the logged-in customer (test IT-05)
NFR-03	Security	No secrets in the code	Secret key in a git-ignored `config.py`
NFR-04	Integrity	Business rules hold even if a page has a bug	Constraints and 7 triggers in the database
NFR-05	Accuracy	Bills are correct to the paisa	Rounding to 2 decimals; unit tests
NFR-06	Auditability	Issued bills never change	Charges copied onto the bill (test IT-14)
NFR-07	Performance	Pages load in under a second with 1,000 customers	Indexes on the busy columns
NFR-08	Usability	A new employee can enter a reading without training	One task per page, live feedback, plain error messages
NFR-09	Portability	Runs on Windows, Linux and macOS	Python + Flask only; SQLite is built in
NFR-10	Mobile	Usable on a phone in the field	Layout adapts below 800 px
NFR-11	Reliability	A failed action leaves nothing half-saved	One transaction per action, rolled back on error

6.3 Prioritisation

We used MoSCoW. Must items are the minimum system (reading → bill → payment, access control, complaints), and we built all of them first. Out of 25 functional requirements: 12 Must (all done), 8 Should (7 done), 4 Could (1 partly done) and 1 Won't for now.

7. Requirements Model: Use Cases and User Stories

There are three actors. "Employee" covers the meter reader, billing officer and technician, since they share one login role. Generating a bill is never started by a person; it is part of recording a reading.

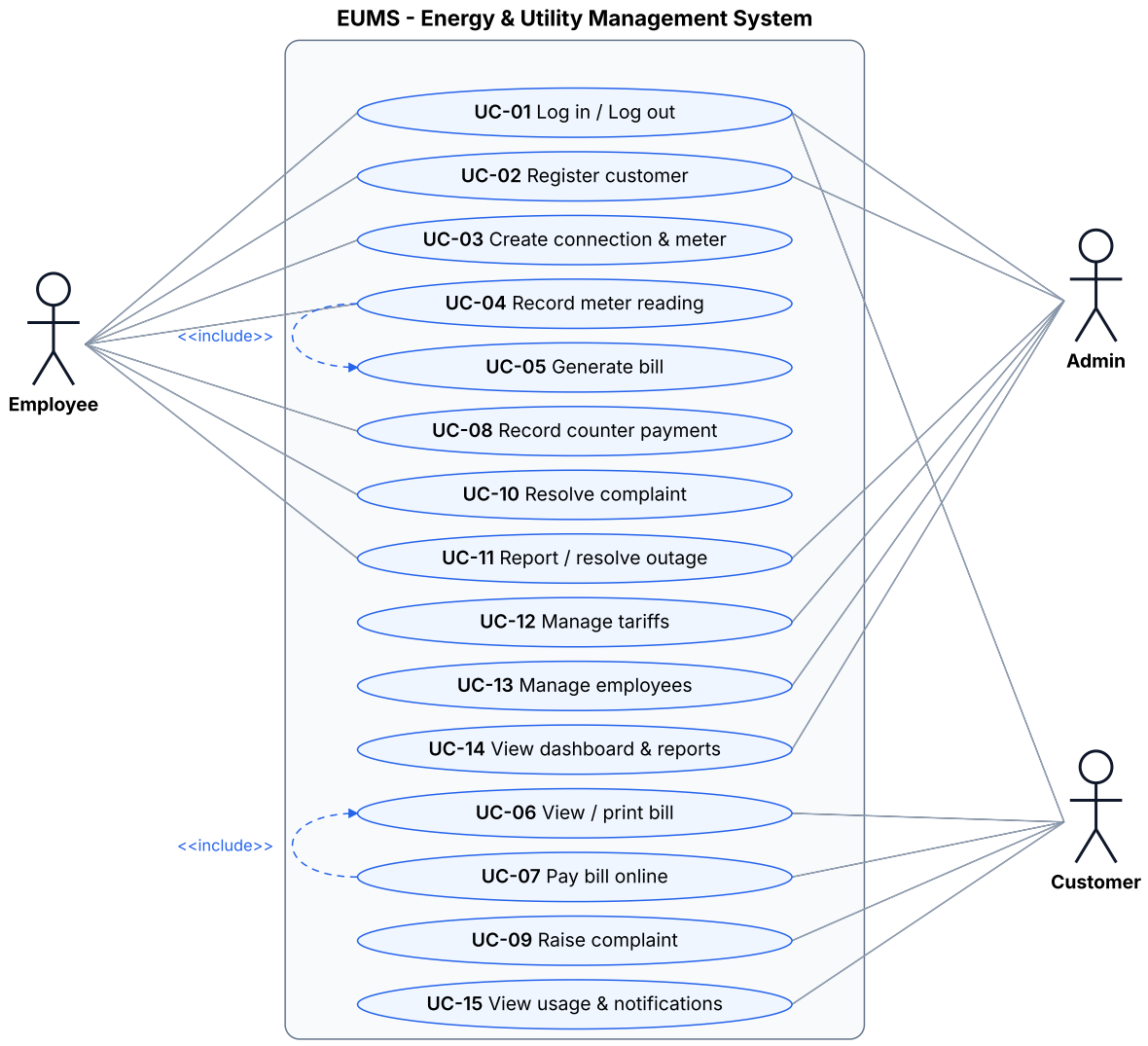


Figure 2: Use-case diagram

UC-04 Record meter reading (includes UC-05 Generate bill)	
Actor / pre-condition	Employee, logged in; connection and meter are active
Main flow	1. Open Add Reading. 2. Pick a meter; the system fills in the customer, utility and last reading. 3. Type the new reading; consumption shows straight away. 4. Enter the date and save. 5. The system saves the reading, works out the bill from the tariff slabs, saves it and notifies the customer. 6. The new bill opens.
Exceptions	Reading lower than the last one: error shown and Save disabled; the database trigger also blocks it. Date not after the last reading: blocked. Same date twice: blocked.
Post-condition	Reading and bill saved; customer gets a "New bill generated" notification

The other use cases (pay bill, complaints, outages, login) are written out in `02_Requirements/Use_Cases_or_User_Stories.pdf`.

7.1 User stories

Story	Acceptance criteria	Status
As a meter reader, I want to see the last reading when I pick a meter, so I don't type the wrong number.	Last value and date shown; consumption updates while typing; lower values block Save.	Done
As a billing officer, I want bills worked out from the slabs automatically.	250 kWh on the domestic plan gives ₹1,207.50 with two slab lines, fixed charge and 5% tax.	Done
As a customer, I want to pay by UPI or card instead of going to the office.	Receipt number generated; bill turns Paid; paying more than due is refused.	Done
As a customer, I want to hear about outages in my area.	Everyone in that area with that utility is notified; nobody else is.	Done
As an admin, I want to change tariff rates without changing old bills.	New rates apply to the next bill only.	Done
As a billing officer, I want to replace a faulty meter without breaking its history.	Old meter marked Replaced; new meter starts from its own reading.	Planned

8. System Design

8.1 Architecture

The system has three layers. The browser shows HTML pages, Flask route functions handle requests and run the business logic, and SQLite stores everything. Pages are built on the server with Jinja templates, so there is very little JavaScript; the only API call is the one that fetches the last meter reading for the reading form.

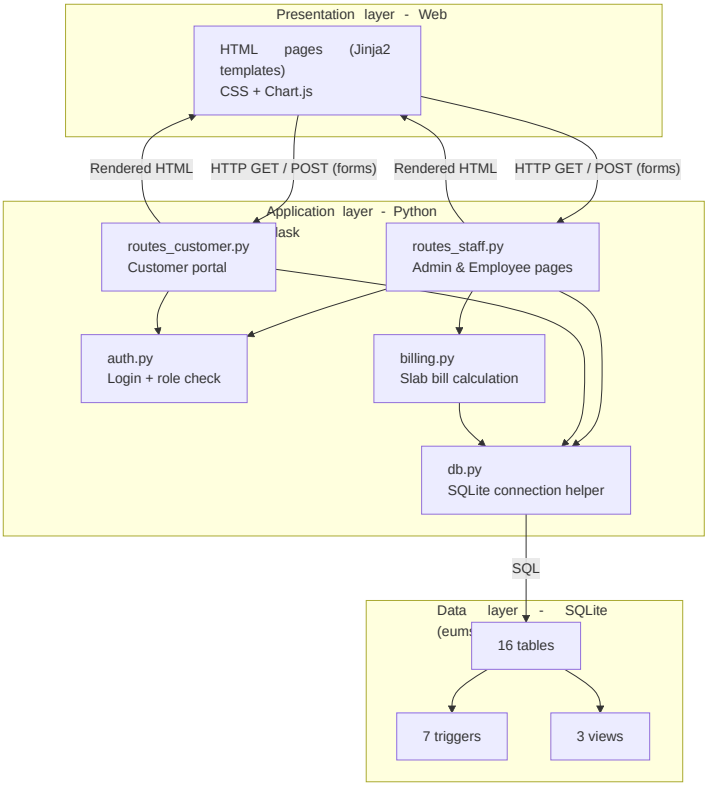


Figure 3: Three-layer architecture

Module	What it does
`app.py` , `auth.py`	Sets up the app, login/logout, and the check that each page is opened by the right role
`db.py`	Opens one database connection per request and closes it after
`billing.py`	Slab bill calculation (a plain function we can unit test) and saving the bill
`routes_staff.py`	All admin and employee pages
`routes_customer.py`	Customer portal; every query is limited to the logged-in customer
`schema.sql` , `seed.py`	Tables, triggers and views; demo data loader

8.2 Database design

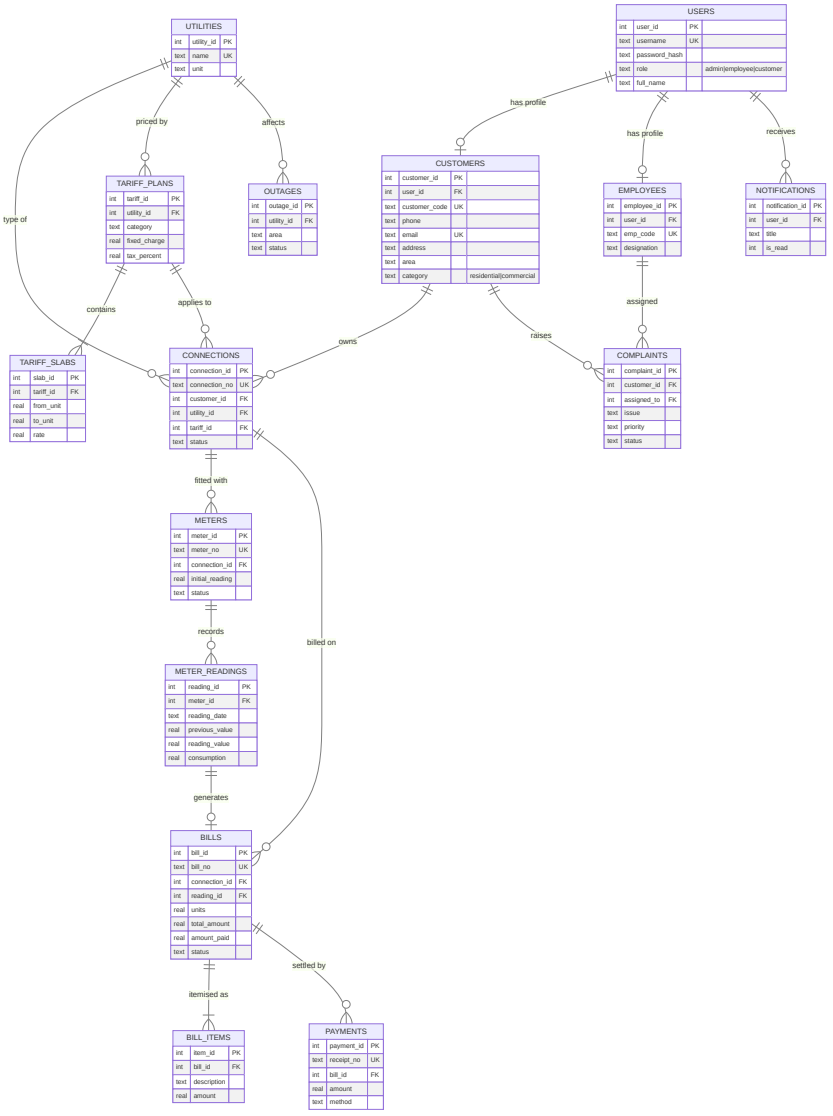


Figure 4: ER diagram (16 tables)

The schema is in 3NF: repeating data such as readings, payments and slabs sits in its own tables, and tariff rates are stored once in `tariff_plans` / `tariff_slabs` instead of on each connection. The one place we broke this on purpose is `bills`, which keeps a copy of the units and charges at the time the bill was made. Tariffs change, but an issued bill should never change.

Trigger / view	Rule it enforces
trg_reading_monotonic, trg_reading_date_order	A new reading can't be lower than the last one or dated before it
trg_no_overpay	A payment can't be more than what is still owed
trg_payment_status	After a payment, mark the bill paid or partial and notify the customer
trg_bill_notify, trg_outage_notify, trg_complaint_status	Send notifications for new bills, outages in the customer's area and complaint updates
v_bill_summary, v_overdue_bills, v_monthly_consumption	Ready-made joins used by the bill pages, overdue list and charts

Billing formula. For each slab: units in slab = max(0, min(U, upper limit) − lower limit). Energy charge = sum of units in slab × rate. Total = (energy + fixed) × (1 + tax%). For 250 kWh on the domestic plan: 100 × 3.50 + 150 × 5.00 = 1,100, plus 50 fixed = 1,150, plus 5% tax = ₹1,207.50.

8.3 Sequence diagram

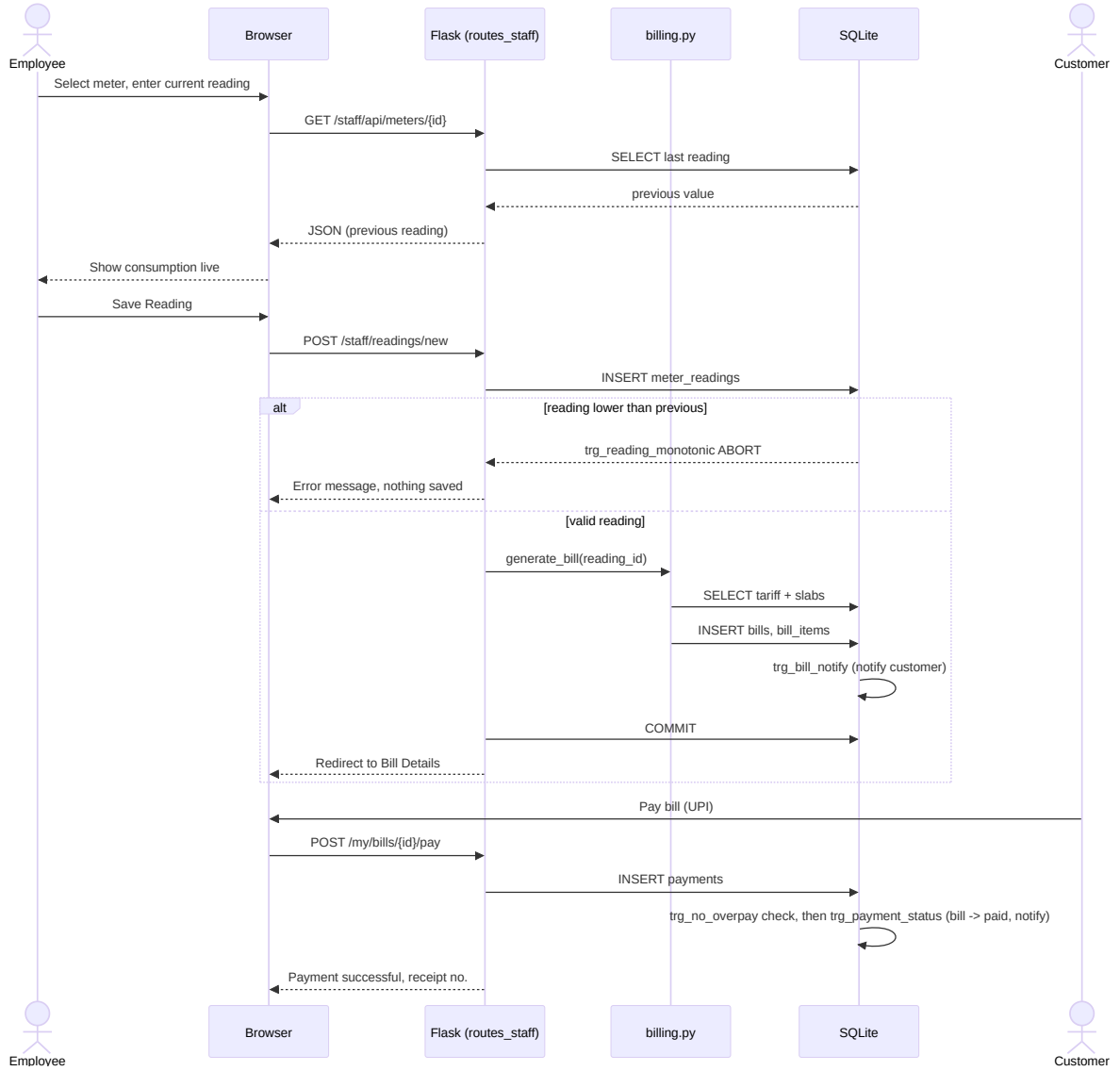


Figure 5: Recording a reading, generating the bill and paying it

The activity diagram, class diagram and data flow diagram are in `03_Design/UML_Diagrams.pdf` and `03_Design/Architecture_Diagram.pdf`. The UI is a plain dark theme with a side menu and one page per feature; all screens are in `03_Design/UI_Screens.pdf`.

9. Technology Stack and Justification

Part	What we used	Why
Language	Python 3	We all learned it in earlier semesters
Web framework	Flask	Small and easy: one function per page, no build step. We found Django too much to learn in the time we had
Database	SQLite	Comes with Python, single file, no server, but still has triggers and views. The SQL can move to MySQL later
Frontend	HTML templates + CSS	No React or Node tooling to learn; pages are rendered by the server
Charts / tests	Chart.js, pytest	One script file for charts; simple test functions
Version control	Git, GitHub	Shared code and a history of who did what

Only Flask has to be installed to run the app.

10. Implementation Progress

The app is roughly 60% done (2,341 lines of code, not counting tests). The end-to-end feature works: an employee enters a reading, the bill is created automatically, the customer pays it online and gets a notification.

10.1 Completed

- Login and role-based access
- Customers, connections and meters
- Meter readings with live checks
- Automatic slab billing, printable bills
- Online, counter and part payments
- Complaints and outages with notifications
- Tariffs, employees, both dashboards
- Customer usage history and profile

10.2 Still in progress

- Reports: date filter and CSV export
- Meter replacement
- Late-payment charges and reminders
- PDF invoice generated on the server
- MySQL version of the schema (for DBMS)
- CSRF protection on forms

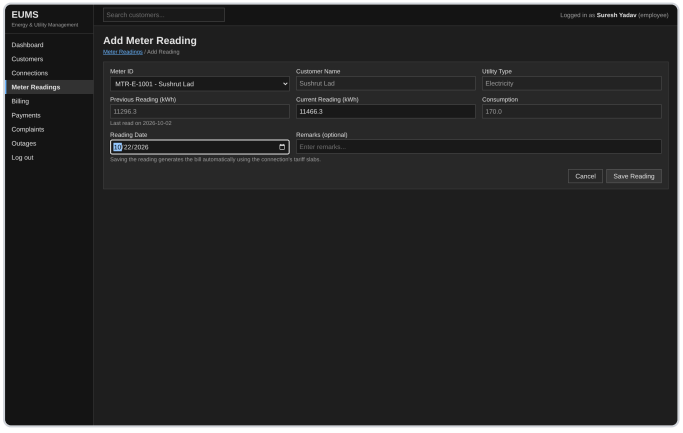


Figure 6: Add Meter Reading (employee)

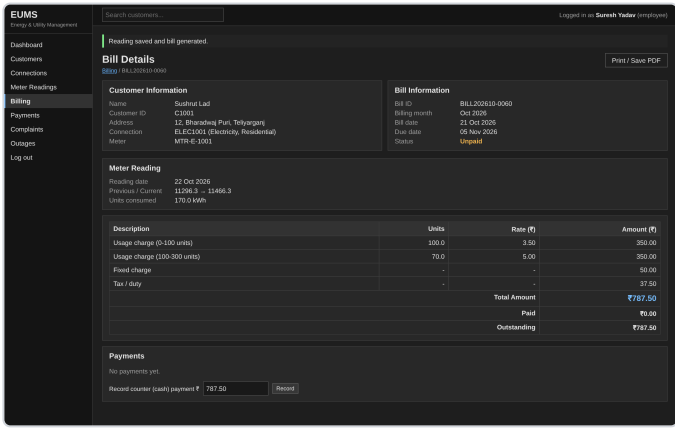


Figure 7: Bill created from the reading



Figure 8: Admin dashboard

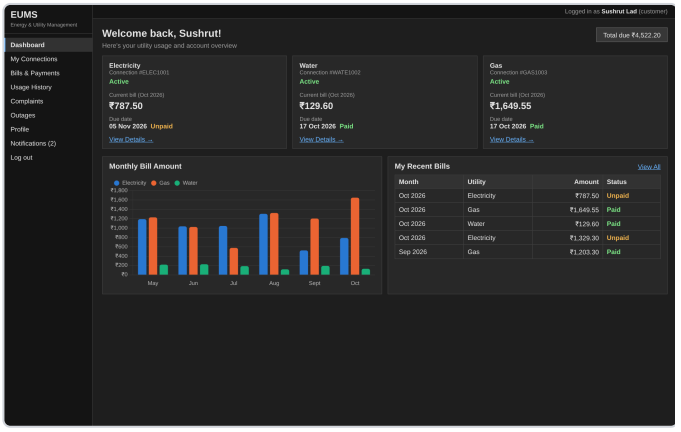


Figure 9: Customer dashboard

10.3 Key implementation decisions

- **Rules live in the database.** Readings going backwards and overpayments are blocked by triggers, so even a bug in a page can't save bad data. The page just shows the trigger's message.
- **Check readings twice.** JavaScript warns the user instantly, and the trigger is the real guarantee.
- **Billing is a separate function**, so we could test it against amounts we worked out by hand.
- **A reading and its bill are saved together** in one transaction; if one fails, neither is kept.
- **Customers never pick an ID in the URL.** Their pages always use the logged-in account, and opening someone else's bill gives a 403.
- **No secrets in the code.** The secret key goes in a `config.py` that git ignores.

11. Testing Performed

We have 6 unit tests for the billing formula and 14 functional tests that click through the real app using Flask's test client, each with a fresh copy of the demo database. Running `python -m pytest -v` gives: **20 passed in 12.29s**. We also checked every page by hand for each role. The full log is in `05_Testing/Test_Evidence/`.

ID	Feature	Input	Expected result	Actual	Status
UT-01	Slab billing	250 kWh, residential electricity (0-100 @3.50, 100-300 @5.00), fixed 50, tax 5%	Energy 1,100.00; tax 57.50; total 1,207.50	As expected	**PASS**
UT-02	Slab billing	0 units	Only fixed + tax = 52.50, no slab rows	As expected	**PASS**
UT-03	Slab billing	Exactly 100 units (slab edge)	Only first slab used; total 420.00	As expected	**PASS**
UT-04	Slab billing	400 units (reaches 300+ slab)	Energy 2,050.00; total 2,205.00; 100 units in top slab	As expected	**PASS**
UT-05	Slab billing	-5 units	ValueError raised	As expected	**PASS**
UT-06	Slab billing	15.5 kL water, no tax	Energy 146.00; total 186.00	As expected	**PASS**
IT-01	Login	admin / admin123	Redirect to /staff/dashboard; dashboard loads	As expected	**PASS**
IT-02	Login	admin / wrong	Stays on login with 'Invalid username or password'	As expected	**PASS**
IT-03	Access control	Open /staff/bills without logging in	Redirect to /login	As expected	**PASS**
IT-04	Access control (RBAC)	Customer opens staff page; employee opens admin-only Tariffs	HTTP 403 for both; employee can open Add Reading	As expected	**PASS**
IT-05	Data isolation	Sushrut (C1001) opens a bill that belongs to Sumit (C1002)	HTTP 403 on bill and pay page	As expected	**PASS**
IT-06	END-TO-END: reading → bill -> payment	Employee adds +250 kWh on Sushrut's meter; Sushrut pays 1,207.50 by UPI	Bill of 1,207.50 created (unpaid); after payment status = paid and 'Payment received' notification	As expected	**PASS**
IT-07	Meter reading validation	Current reading 10 below previous	Trigger error 'Reading cannot be lower...'; no bill created	As expected	**PASS**
IT-08	Meter reading validation	Reading date 2020-01-01	Trigger error 'Reading date must be after...'	As expected	**PASS**
IT-09	Payment validation	Sushrut pays outstanding + 100	Trigger error 'Payment exceeds outstanding balance'; bill not paid	As expected	**PASS**
IT-10	Counter payment	Billing officer pays half the bill in cash	Bill status = partial	As expected	**PASS**
IT-11	Outage alerts	Admin reports electricity outage in Alambagh	Every Alambagh electricity customer notified; Hazratganj customers not notified	As expected	**PASS**
IT-12	Complaints	Sushrut raises a complaint; technician marks it resolved	Complaint visible to staff; status resolved with timestamp	As expected	**PASS**
IT-13	Customer registration	Register Sushrut Lad again with the same email	Error 'already exists'; customer count unchanged	As expected	**PASS**
IT-14	Tariff management	Admin changes slab rate to 9.99	New rate saved; old bill total unchanged	As expected	**PASS**

Still to do: testing by someone outside the team, boundary and basic security checks, a run with 1,000+ customers, and acceptance testing with classmates.

12. Project-Management Status

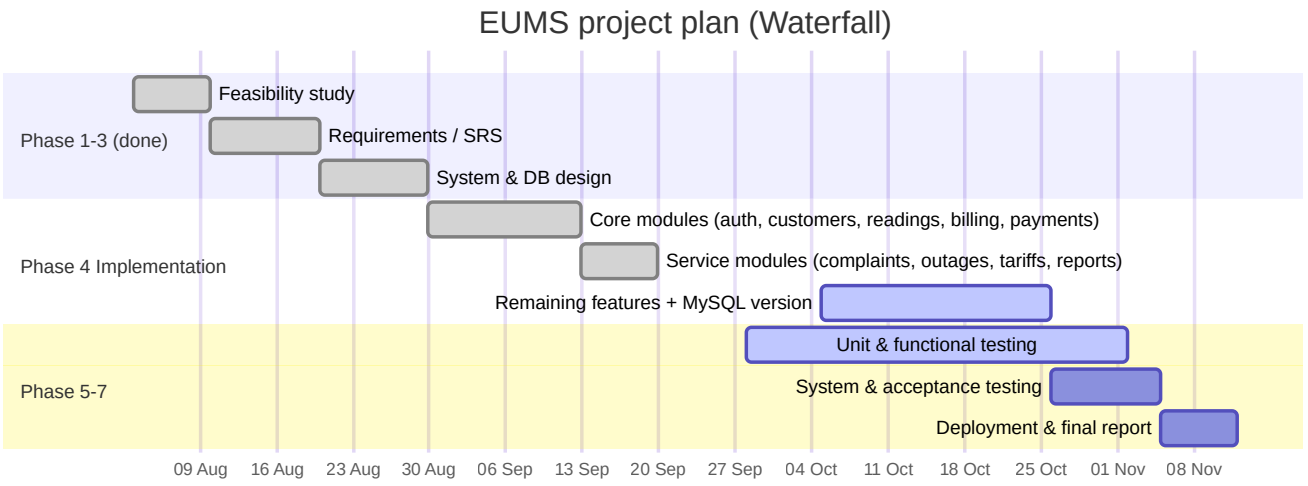


Figure 10: Original plan (Gantt chart)

Phase	Planned by mid-sem	Actual
Feasibility, requirements, design	100%	100%
Implementation	50%	About 60% (19 of 25 requirements, 1 partial)
Testing	20%	20 automated tests, all passing
MySQL version (DBMS)	Started	Not started, moved to weeks 9–10

Delayed or pending: the MySQL port (we chose SQLite first to keep setup simple), report filters and export, meter replacement, late fees and CSRF protection.

Estimate (basic COCOMO, organic mode). We expect about 3.5 KLOC when finished. Effort = $2.4 \times 3.5^{1.05} \approx 8.9$ person-months, duration = $2.5 \times E^{0.38} \approx 5.7$ months. Our real effort is about 2 person-months (four of us part-time), which is much less. COCOMO is tuned for industry projects with heavier documentation and review, so it overestimates something this small. We treat it as an upper limit and cut Could-priority features if we run short.

Risk	Likelihood / impact	What we're doing about it
Team new to web development	High / High	Simplest stack possible; built one page together and used it as the pattern
Wrong bill amounts	Medium / High	Billing unit-tested against hand calculations; bills keep their breakdown
Bad data (lower readings, overpaying)	High / High	Triggers plus form checks, covered by tests
A customer seeing someone else's data	Low / High	Ownership check on every customer page
Finding requirement mistakes late	Medium / Medium	Review at the end of each phase; end-to-end feature built early
Exams clashing with deadlines	High / Medium	Must items first; weekly meeting; swap tasks when someone is busy

13. Individual Contribution

Name	Reg. no.	Main work
Sumit P Surendran	20243590	Database design, ER model, seed data, reports module
Sreethan Reddy	20243550	Frontend pages, customer portal, UI design
Sushrut Lad	20243551	Backend (billing, readings), database triggers
Viswajith	20243559	Requirements/SRS, testing, documentation

14. Challenges Faced and Solutions Attempted

- **Slab billing was easy to get wrong** at the slab edges and with decimals. We wrote it as one small function and tested edge cases.
- **Readings going backwards** if someone skips the form. We added triggers so the database itself refuses them.
- **Charts mixing kWh, kL and gas units.** Comparison charts use rupees; usage charts are drawn per utility.
- **Tests affecting each other.** Each test now gets its own fresh database file.

Appendix: AI-Tool Disclosure and How to Run

Used for	What AI did
Documentation and diagrams	We used AI to put our notes into proper formal language for the report and other documents, and to draw the diagrams neatly.
User interface	The web pages were made with AI, starting from existing templates.

```
cd 04_Source_Code
pip install -r requirements.txt
python seed.py          # creates the demo database
python app.py           # open http://127.0.0.1:5000
python -m pytest -v     # runs the 20 tests
```